ZYVORAI LABS

PacketWolf

Customer Feature Guide

2026-10-28

zyvor.dev · info@zyvor.dev

> **Kernel-native network intelligence for Kubernetes.**

PacketWolf turns a Cilium/eBPF cluster into a single pane of glass that sees which process made which network call, learns normal traffic, and detects and contains threats before they spread. It fuses kernel-level process attribution, Hubble flow visibility, AutoPolicy learning, and an agentic network copilot into one Rust platform with a web console (light and dark, desktop and phone), an 18-tab TUI, and the netpred CLI. Where other tools stop at pod-to-pod, PacketWolf goes process-to-socket and cross-layer for real root cause and autonomous healing.

400+ documented REST routes (generated OpenAPI) · **90** console pages and **60** embeddable panels · **10** kernel intelligence modules · **15** intelligence engines · **6** Kubernetes CRDs · **<1%** CPU overhead

This is the customer-facing onboarding guide — how to access the product, your first workflows, and how to use every feature. A print-ready PDF of the same content sits alongside this file. Screenshots are taken from the current console against synthetic sample data (invented namespaces and workloads), not from a customer cluster. This edition describes the current PacketWolf build; the public evaluation image on GitHub Container Registry is updated at each release, so check its release notes for the tag that carries a given feature.

Contents

- What is new in this edition
- Getting started — access & first workflows
- Security model — who can see and do what
- Kernel Intelligence
- eBPF Data Plane
- Flow Visibility & Topology
- Threat Detection
- Policy & Zero Trust
- Autonomous Response
- Ask Zyra Copilot
- Reliability Engineering
- Compliance, Cost & Prediction
- Surfaces & Platform
- Container & Workload Consoles
- Known limits
- Screenshots

What is new in this edition

The console, the access model and the API were reworked. What changed for you:

- **A new console, in the Netra style.** Light and dark themes, a top navigation bar (Overview, Investigate, Diagnostics, Security, Intelligence, Reports, Settings; a **More** menu on phones), pages built from the same cards, tiles, tabs and tables, and a **search palette** (`⌘K` / `Ctrl+K` , or `/`) that jumps to any page you are allowed to open. Every route has its own page title, and focus moves to the page heading on navigation.
- **Phone layout.** KPI tiles form a two-column grid, tabs and filter chips scroll sideways in one row, tables scroll with a pinned first column, page-header housekeeping folds into one overflow button, and tap targets are at least 44 px. Layouts are checked at 320, 390, 1024 and 1440 px.
- **Honest numbers.** Missing or stale data shows a dash or *Not available*, never a false 0, 100% or green status. Large numbers are compacted (`1.8B`), tiny values keep their unit, timestamps can show relative time, and a failed request (4xx or 5xx) is shown as an error, never as an empty list.
- **Namespace-scoped accounts.** An account can be confined to one or more namespaces. The server enforces it on every route; the console shows a namespace switcher, hides cluster-wide pages and says why when something is not available to a scoped account. See Security model.
- **Policy approvals with separate duties.** Submitting a policy for approval needs the *Simulate* capability; approving or rejecting needs *Approve* and the approval's namespace in scope. Each approval records who submitted and who reviewed it, and every step is audited.
- **Safer sessions.** Logout is server-side (the token is revoked and open sockets are closed), sessions have an absolute lifetime (7 days by default), disabling or re-scoping an account takes effect on tokens already issued, logins are locked out after repeated failures, and shells are capped and idle-limited.
- **Hardened request handling.** Every `kubectl` call the API makes passes one argument guard, outbound webhooks and SSO fetches refuse private and metadata addresses unless you opt in, and forwarded client addresses are honoured only from proxies you list.
- **A complete API reference.** `docs/openapi.yaml` and `/api-docs/openapi.json` document every route with its tag, required role or capability, parameters and standard error responses; both are generated from the router, so they cannot drift.
- **Install defaults.** The Helm chart generates a random admin password and API key on first install, trusts no caller by default, bundles a pinned `kubectl` , and makes node maintenance (cordon, drain) an explicit opt-in.

Getting started

How to access it

- **Web:** the console is served by the API (port 9191 in the container image; `https://:9443` on a remote host deploy; through your ingress in a cluster). Keyboard: `⌘K` / `Ctrl+K` opens the search palette, `/` opens global search, `?` opens Help, `r` refreshes, and `g h` / `g f` / `g t` / `g p` / `g a` / `g c` / `g s` jump to Dashboard / Flows / Topology / Policies / Anomalies / Compliance / Settings (the shortcuts list is in Help). Navigation, the palette and the shortcuts only offer pages your role and namespace scope allow.
- **CLI:** `netpred` — ten subcommands for scripting network intelligence end to end. Examples: `netpred kernel connections` (TCP lifecycle attribution), `netpred explain` (cross-layer root cause), `netpred trace --namespace kube-system --verdict DROPPED`, `netpred policy suggest`, `netpred graph show`, `netpred chaos run latency-spike --target my-pod`. `netpred tui` launches the terminal UI; `netpred --help` lists everything.
- **API:** REST + WebSocket web-api mounted under `/api/v1` (400+ documented routes). Health at `GET /health`; interactive docs at `/swagger-ui`; machine spec at `/api-docs/openapi.json` (the same document as `docs/openapi.yaml`, generated from the router, with the minimum role or capability of every route). Example: `curl -s -H "X-Api-Key: $KEY" https://<host>:9443/api/v1/kernel/processes`. Live streams over WebSocket, e.g. `wscat -c wss://<host>:9443/api/v1/ws/flows/live`.
- **Login:** Auth is on by default and nothing ships with a fixed password or key. Sign in as `admin` with the password generated at install (Helm: read it from the release Secret, key `PACKETWOLF_ADMIN_PASSWORD`; the chart's install notes print the command), with a local, LDAP, OIDC or SAML account, or authenticate scripts with the admin API key (`X-Api-Key`) or a JWT bearer token (`JWT_SECRET`, min 32 chars). Signing out revokes the session on the server. WebSocket auth is passed via the `Sec-WebSocket-Protocol` header, never the URL. `AUTH_DISABLED=true` exists for local development only and is refused in production.
- **Needs:** A Kubernetes cluster (K3s / EKS / GKE / any distro) running Cilium 1.14+ with Hubble enabled — PacketWolf can auto-install Cilium and auto-enable Hubble if they are missing. The container image is Linux x86_64.

Your first workflows

- **Root-cause a broken service (God Mode)**
  1. Run `netpred kernel connections` to see which process/pod owns each socket and spot stuck or churning connections.
  2. Run `netpred explain payment` to fuse kernel, DNS, network, and policy state into one cross-layer root-cause narrative.
  3. Open the web dashboard `/rootcause` view to review dropped-packet-to-policy correlation and apply a one-click corrective CiliumNetworkPolicy fix.

- **Learn and enforce a least-privilege policy**
  1. Let AutoPolicy observe traffic, then draft a policy with `netpred policy suggest` or `GET /api/v1/network/policy-recommendation`.
  2. Simulate impact with `netpred policy simulate my-policy.yaml` or `POST /api/v1/network/blast-radius` to see flows and services affected.
  3. Advance the workload through the Zero-Trust Pilot (`/embed/zero-trust-pilot`) observe → simulate → approve flow, or apply via the visual `/rule-builder`.
- **Detect and contain a threat**
  1. Review ThreatSense detections at `GET /api/v1/network/threats` (confidence, reason chain, recommended action) and the correlated `/embed/threat-threads` attack story.
  2. Pull `GET /api/v1/network/response/recommendations` to map the detection to a containment profile with a simulated blast radius.
  3. Preview containment with `POST /api/v1/network/containments/simulate`, then `apply` (or `approve` a gated response) — enforced via the WorkloadContainment CRD with auto-rollback on failure.
- **Validate resilience with chaos and canary**
  1. Inject a real tc-netem fault with `netpred chaos run latency-spike --namespace default --target my-pod` (circuit-breakered, auto-cleanup).
  2. Shift traffic progressively with `netpred canary start my-deploy --weight 10`, watching success rate.
  3. Promote with `netpred canary promote my-deploy` (auto-promotes at 99% success, auto-rolls back below 90%).
- **Deploy to a Cilium cluster and open the dashboard**
  1. Install with the Helm chart from your deployment package: `helm upgrade --install packetwolf ./chart -n packetwolf-system --create-namespace --set api.persistence.enabled=true`. Credentials are generated on first install and kept across upgrades; behind an ingress also set `api.trustedProxies` to the ingress controller's pod or node CIDR so login lockout and rate limits see real client addresses.
  2. Verify: `kubectl -n packetwolf-system rollout status deployment/packetwolf-api` and `GET /health`.
  3. Open the console through your ingress (or `kubectl port-forward`) and sign in as `admin`.

Security model

PacketWolf is deny-by-default: every request needs a credential, and what a credential may see or do is decided on the server, not in the browser. The console only mirrors those decisions.

Roles and capabilities. `viewer` and `partner` read. `operator` can also run simulations and apply low-impact containment. `approver` approves. `security-admin` holds simulate, contain, approve and break-glass. `platform-admin` configures integrations. `admin` can do everything. Role names are accepted in any case and separator spelling, and an unknown role is refused on every route.

Action	Needs
Read flows, policies, maps	any role, within your namespaces
Simulate a policy or containment, submit a policy for approval	`Simulate` (operator, security-admin, platform-admin, admin)
Approve or reject a submitted policy	`Approve` (approver, security-admin, admin) and the approval's namespace in your scope
Apply a policy directly, release, rollback, quarantine, governance	unscoped `admin`
Users, SSO settings, node maintenance, `/metrics`	unscoped `admin`

Namespace-scoped accounts. Give an account a list of namespaces (Security, User Accounts) and the server confines it:

- Flow-derived views (flows, service map, DNS, drops, latency and so on) show a flow when its source **or** destination is in scope. A single object outside the scope reads as not found; naming a namespace outside the scope is refused with 403.
- Cluster-level data that cannot be split by namespace (Cilium internals, nodes, SLOs, alert channels, containers, multi-cluster, replay and similar) is refused with 403 for a scoped account, and the console hides those pages.
- A scoped account with several namespaces works in one at a time (the namespace switcher in the console); with exactly one, it is applied automatically.
- The namespaces claim wins over the role: an `admin` limited to namespaces is a scoped caller, not a platform admin.
- Role and namespace changes reach tokens already issued: for local and SSO-provisioned accounts, every request re-reads the account, so a demotion or a narrower scope applies on the next request and open WebSockets are closed. LDAP identities keep the role they had at sign-in for the life of the token.
- Embed tokens and partner keys are view-only; confine them to namespaces with `api.embedTokenNamespaces` and `api.partnerNamespaces` (Helm) so a leaked embed URL cannot read the whole cluster.

Approvals. A submitted policy must be a single `CiliumNetworkPolicy` with a valid name and a namespace equal to the approval's; size limits apply and YAML anchors, aliases and tags are refused. The policy is checked again before it is applied on approval. Each approval records the submitter and

reviewer (name, kind of credential, role) and its namespace, and submit, approve, reject and direct apply are written to the audit log.

Audit. State-changing actions are logged with time, actor, action, target and outcome and kept for 30 days. Actions PacketWolf records but does not perform (see Known limits) are audited with outcome `recorded`. Reading the audit log does not write entries.

Sessions. Logout revokes the token server-side (shared across API replicas when Redis is configured). Sessions have an absolute lifetime (default 7 days) and cannot be extended by refreshing. Five failed sign-ins for the same address and username start a lockout that doubles up to 15 minutes, and a separate per-username limit stops guessing from rotating addresses. Interactive shells (pod, container, VM console) are audited, capped (3 per user, 20 total by default), idle-limited and need a real admin credential, never source-address trust alone.

Behind a proxy. `X-Forwarded-For` is honoured only from proxies listed in `PACKETWOLF_TRUSTED_PROXY_IPS` (Helm `api.trustedProxies`). Without it everyone behind the ingress shares one lockout and rate-limit bucket.

Requests and webhooks. Names that start with `-` are refused so a request value cannot become a `kubectl` option, every `kubectl` invocation passes one guard, errors from Kubernetes are sanitised before reaching clients, and outbound webhooks and SSO fetches refuse private, loopback and cloud-metadata addresses unless you set `PACKETWOLF_ALLOW_PRIVATE_WEBHOOKS` or `PACKETWOLF_ALLOW_PRIVATE_IDP`.

Containers. The chart runs as non-root with a read-only root filesystem and all capabilities dropped, ships NetworkPolicies for Redis and the API, and keeps node maintenance (cordon, drain) off unless you set `rbac.nodeMaintenance.enabled=true`.

> Cluster-network trust (callers inside the cluster treated as admin without a credential) is **off** in the chart's default install. Turn it on (`api.env.trustClusterNetworks=auto`) only on a cluster you trust, set the pod and service CIDRs, and restrict who can reach the API with NetworkPolicy.

1. Kernel Intelligence

See past pod-to-pod to the exact process behind every socket.

- **Process-to-Network Attribution** — Maps every network socket to the exact process, container, and pod that owns it via /proc/net and cgroup resolution. — *Answer 'which process made which call' instead of guessing from pod IPs.*
  - **How:** CLI: `netpred kernel processes`. API: `GET /api/v1/kernel/processes`. Web UI: Kernel (`/kernel`).
- **TCP Lifecycle Tracking** — Follows connections through SYN, ESTABLISHED, FIN, RST, and TIME_WAIT with per-connection timing. — *Spot stuck, half-open, and churning connections at a glance.*
  - **How:** CLI: `netpred kernel connections`. API: `GET /api/v1/ebpf/conntrack`. Web UI: Conntrack (`/conntrack`).
- **Behavioral Fingerprinting** — Builds per-workload statistical profiles of connection patterns, port usage, and timing for drift detection. — *Catch a workload behaving unlike its own history, not just against static rules.*
  - **How:** CLI: `netpred kernel fingerprint`. API: `GET /api/v1/kernel/fingerprints`.
- **Cross-Layer Correlation** — Links L3, L4, L7, process, and eBPF signals into one unified view of an incident. — *Stop pivoting between five tools to reconstruct a single event.*
  - **How:** CLI: `netpred kernel correlate`. TUI: Kernel tab (`k` to switch sub-views). API surfaces correlation via `GET /api/v1/kernel/events`.
- **eBPF Syscall Tracing** — Traces connect, bind, accept, sendmsg, and execve via kprobes and tracepoints (Tier 2, needs CAP_BPF). — *Watch network syscalls as they happen at kernel speed.*
  - **How:** CLI: `netpred kernel syscalls`. Enable custom probes with `NETPRED_EBPF_AUTO=1` (needs CAP_BPF / BTF); drained by `netpred operator`.
- **God Mode Explain** — netpred explain combines kernel, DNS, network, and policy state into one root-cause narrative. — *One command replaces an hour of cross-layer detective work.*
  - **How:** CLI: `netpred explain` (also `netpred kernel godmode` for the full kernel view). TUI: Kernel/RootCause tabs.

> Golden feature: netpred explain payment returns a full cross-layer root cause in seconds.

2. eBPF Data Plane

Decoded kernel maps and custom probes — honest about what is live.

- **Decoded Cilium Maps** — Reads Cilium's pinned conntrack, policy, LB, ipcache, and drop maps and decodes them into structured, Kubernetes-enriched rows. — *Read real kernel state without deciphering raw hex.*
  - **How:** API: `GET /api/v1/ebpf/conntrack`, `/ebpf/policy-map`, `/ebpf/ipcache`, `/ebpf/lb`. Web UI: eBPF (`/ebpf`), Policy Map (`/policy-map`).
- **Kubernetes Map Enrichment** — Joins map entries to CiliumEndpoints so policy keys, conntrack tuples, and LB backends resolve to real pods and services. — *Every kernel row names the workload it belongs to.*
  - **How:** API: the same `/api/v1/ebpf/*` endpoints return `kubernetes` / `src_kubernetes` / `dst_kubernetes` enrichment blocks (RBAC-scoped). Web UI: Conntrack (`/conntrack`).
- **Custom NetPredator Probes** — Opt-in BPF phases for TCP lifecycle, process map, DNS path, drop monitor, and TCP anomalies drain a ring buffer into the event bus. — *Add your own kernel telemetry beyond what Cilium exposes.*
  - **How:** Build with `make ebpf-bpf`, enable with `NETPRED_EBPF_AUTO=1` (needs CAP_BPF), then run `netpred operator` (or the TUI) to drain the EVENTS ringbuf into the EventBus.
- **Honest Data-Plane Status** — The /ebpf-dataplane view reports exactly which probe phases attached, which failed, and which remain userspace. — *At 2 a.m. you see 'phase 4 failed to attach' instead of an empty dashboard.*
  - **How:** Web UI: eBPF Data Plane (`/ebpf-dataplane`). API: `GET /api/v1/ebpf/dataplane`.
- **Drop Analytics** — Aggregates dropped-packet counters by reason category with trend lines. — *Turn silent kernel drops into a ranked, explained list.*
  - **How:** API: `GET /api/v1/network/drop-decoder`. Web UI: Drops (`/drops`), Drop Decoder (`/embed/drop-decoder`).

> PacketWolf splits production-grade pinned-map readers from opt-in custom probes and labels each honestly — no dashboard placeholders.

3. Flow Visibility & Topology

A live map of who talks to whom, at what cost, over which path.

- **Live Flows** — Streams Hubble flows with verdicts, drops, and L7 context through the web UI and TUI. — *Watch real cluster traffic instead of sampled snapshots.*
  - **How:** CLI: `netpred trace --namespace --verdict DROPPED`. API: `GET /api/v1/flows`; live via `ws:///api/v1/ws/flows/live`. Web UI: Flows (`/flows`).
- **Service Map & Graph Engine** — A petgraph dependency graph updates in real time with traffic volume and latency, supporting shortest-path and impact queries. — *Know instantly what breaks if a service goes down.*
  - **How:** CLI: `netpred graph show`, `netpred graph query --from frontend --to backend`. API: `GET /api/v1/graph/topology`. Web UI: Service Map (`/servicemap`), Graph Explorer (`/graph-explorer`).
- **Top Talkers & Latency Profiler** — Ranks bandwidth and connection hotspots and surfaces P50/P99 latency per service edge. — *Find the noisy neighbor and the slow hop without hunting.*
  - **How:** API: `GET /api/v1/network/top-talkers`, `GET /api/v1/network/latency-profiler`. Web UI: `/embed/top-talkers`, `/embed/latency-profiler`.
- **DNS Wolf** — Per-workload DNS query volume, failures, resolvers, and tunneling signals. — *See which pod is hammering DNS or leaking data over it.*
  - **How:** CLI: `netpred kernel dns`. API: `GET /api/v1/network/dns-wolf`. Web UI: DNS Wolf (`/embed/dns-wolf`), DNS (`/dns`).
- **mTLS Visibility** — Maps mTLS coverage and gaps across service-to-service traffic. — *Prove which flows are actually encrypted, not just assumed to be.*
  - **How:** API: `GET /api/v1/network/mtls-visibility`. Web UI: mTLS Visibility (`/embed/mtls-visibility`).
- **VM Network Context** — Extends flow and access visibility to KubeVirt and libvirt/Machina virtual machines alongside pods. — *One network view spans containers and VMs together.*
  - **How:** API: `GET /api/v1/network/vm-access-ai`. Web UI: VM Access AI (`/embed/vm-access-ai`).

4. Threat Detection

Learned, temporal detection that stays quiet until something is genuinely wrong.

- **ThreatSense Detectors** — A tunable rule registry emits detections with confidence, reason chains, and a recommended action. — *Alerts arrive with the 'why' and the 'what next' attached.*
  - **How:** API: `GET /api/v1/network/threats`; tune via `GET /api/v1/network/rules` and `PATCH /api/v1/network/rules/{id}`. Web UI: Incidents (`/incidents`).
- **Learned Behavioral Baselines** — Per-workload EWMA baselines score the current window with a z-score against the workload's own history, gated at 2.5 sigma. — *Detection is learned, not a brittle static count threshold.*
  - **How:** API: `GET /api/v1/network/flow-baselines`; learned scores fold into `GET /api/v1/network/threats`.
- **NXDOMAIN-Storm / DGA Detection** — Flags workloads with high DNS failure ratios and many distinct failed names as domain-generation-algorithm C2 beaconing. — *Catch malware phoning home through algorithmic domains.*
  - **How:** First enable L7 DNS visibility: `PUT /api/v1/workloads/{id}/deep-inspection`. Detections surface in `GET /api/v1/network/threats` (rule-006, `nxdomain_rate > 50%`).
- **Beaconing Detection** — Identifies command-and-control beacons by inter-arrival periodicity (coefficient of variation), not traffic volume. — *Spot low-and-slow C2 that volume thresholds miss.*
  - **How:** API: `GET /api/v1/network/threats` (ThreatSense periodicity detector). Web UI: Incidents (`/incidents`).
- **ThreatThread & Attack Graph** — Correlates alerts into multi-stage attack stories and unified attack paths across risky workloads. — *See the whole kill chain, not scattered single alerts.*
  - **How:** API: `GET /api/v1/network/threat-threads`, `GET /api/v1/network/attack-graph`. Web UI: `/embed/threat-threads`, `/embed/attack-graph`.
- **GeoThreat Enrichment** — Adds ASN, country, and reputation context to flows, with optional MaxMind and threat-intel feeds. — *Know when traffic is heading somewhere it never should.*
  - **How:** API: `GET /api/v1/network/geo-threat`. Web UI: `/embed/geo-threat`. Optional feeds via `PACKETWOLF_GEOIP_DB_PATH` and `PACKETWOLF_THREAT_INTEL_PATH`.
- **ProcessLens & RuntimeGuard** — Correlates Tetragon exec events to Hubble flows and surfaces runtime exec anomalies. — *Tie a suspicious process launch to the connection it opened.*
  - **How:** API: `GET /api/v1/network/process-lens` (or `GET /api/v1/flows?enrich_process_lens=true`), `GET /api/v1/network/runtime-guard`. Web UI: `/embed/runtime-guard`.

> L7 detectors stay inert on L3/L4-only clusters — no false alarms until deep inspection provides the needed DNS and HTTP visibility.

5. Policy & Zero Trust

Learn traffic, draft least-privilege rules, and simulate before you enforce.

- **AutoPolicy Learning** — Learns communication patterns from conntrack and generates CiliumNetworkPolicy YAML with a 7-feature ML confidence score. — *Stop hand-writing policies from guesswork.*
  - **How:** CLI: `netpred policy suggest`. API: `GET /api/v1/network/policy-recommendation`, apply with `POST /api/v1/modules/autopolicy/apply`. Web UI: AutoPolicy (`/autopolicy`).
- **Visual Rule Builder** — Form-based policy creation with schema validation, conflict detection, simulate, and apply — no YAML required. — *Build correct network policy without editing manifests.*
  - **How:** Web UI: Rule Builder (`/rule-builder`), Visual Policy (`/policy-visual`), Policy Editor (`/policy-editor`). Validate via `POST /api/v1/policies/validate-networkpolicy`.
- **Guided Scope Wizard** — Pick a VM, Pod, Namespace, or the whole cluster through a use-case → target → review wizard instead of hand-typing labels or namespaces — the same scope model powers Containment's "Guided setup" and the Guard console's "Guided peer" picker for RuleSet rules. — *Scope a rule or a containment action correctly on the first try.*
  - **How:** Web UI: Containment (`/containment`) → Guided setup; Rules (`/rules`) → a workload's Baseline/RuleSet tab → Guided peer on any rule's Peer field. API: `POST /api/v1/network/packetwolf-rulesets/{simulate,apply}` accepts an optional `ruleScopes` map resolved server-side into the rule's peer selector.
- **Zero-Trust Pilot** — Guided observe → simulate → approve workflow that ratchets a workload toward least privilege. — *Reach default-deny safely, one reviewed step at a time.*
  - **How:** API: `GET /api/v1/network/zero-trust-pilot/preview`, `POST /api/v1/network/zero-trust-pilot/advance`. Web UI: `/embed/zero-trust-pilot`.
- **Policy Insight** — Explains any policy in plain language and scores it on a least-privilege scorecard. — *End the 'who wrote this NetworkPolicy and why' archaeology.*
  - **How:** API: `POST /api/v1/policies/insight` (explain + score), or `POST /api/v1/policies/explain` and `POST /api/v1/policies/score` separately. Web UI: Policies (`/policies`).
- **Policy Approvals** — A four-eyes queue for network-policy changes: an operator submits a `CiliumNetworkPolicy` (Simulate capability), a reviewer with the Approve capability compares the diff and approves or rejects it, and approval applies the policy. Each entry shows its namespace, submitter and reviewer; a policy that fails to apply stays pending with the server's message. Scoped accounts see and decide only their namespaces' approvals. — *Separate who proposes a change from who lets it ship.*
  - **How:** Web UI: Policy Approvals (`/policy-approvals`, shortcut `g v` in the embedded console). API: `GET /api/v1/network/approvals` (pending only), `POST /api/v1/network/approvals/{id}/approve|reject`, submit with `POST /api/v1/network/actions/submit-approval`. Optional notification: `PACKETWOLF_APPROVAL_WEBHOOK_URL`.

- **Blast-Radius Simulation** — Models exactly which flows and services a policy change would affect before it ships. — *Never break production discovering what a rule blocks.*
  - **How:** CLI: `netpred policy simulate my-policy.yaml`. API: `POST /api/v1/network/blast-radius`. Web UI: `/embed/blast-radius`.
- **Policy Drift Scanner** — Ranks cluster-wide drift between intended and live policy posture. — *See where reality has diverged from your security intent.*
  - **How:** API: `GET /api/v1/network/policy-drift-scan`. Web UI: Policy Drift (`/embed/policy-drift`).
- **KubePosture Scoring** — Computes a segmentation score and flags default-deny gaps and world-egress findings. — *Get a single number for how locked-down the cluster really is.*
  - **How:** API: `GET /api/v1/network/kubeposture`. Web UI: KubePosture (`/embed/kubeposture`).

6. Autonomous Response

Detect, explain, simulate, contain, and roll back — with a human in the loop by default.

- **Self-Healing Engine** — Reads eBPF drop counters to diagnose policy denials, unreachable backends, MTU, and DNS failures and proposes targeted fixes. In the web console **PacketWolf does not run a fix**: the button reads *Record fix as handled*, asks for confirmation, and only writes an acknowledgement (`status: recorded`) to the audit log; you apply the fix yourself. — *Turn silent breakage into an actionable, reviewable fix.*
  - **How:** API: `GET /api/v1/network/healer-auto/preview` , `POST /api/v1/network/healer-auto/apply` . Web UI: Healer (`/healer`), `/embed/healer-auto` . TUI: Healer tab (`d` to detect, `a` to apply).
- **Detect-to-Contain Loop** — Maps each high/critical detection to a containment profile, simulates blast radius, and computes a gated response confidence. — *Response is pre-vetted for safety before anyone clicks apply.*
  - **How:** API: `GET /api/v1/network/response/recommendations` . Web UI: Incidents (`/incidents`).
- **Gated Auto-Response** — When enabled and blast-radius-safe, confident responses auto-apply with verify and automatic rollback on failure; otherwise they queue for approval. — *Contain fast-moving threats without risking a self-inflicted outage.*
  - **How:** Enable with `PACKETWOLF_AUTO_RESPONSE_ENABLED=true` (tune `PACKETWOLF_AUTO_RESPONSE_CONFIDENCE` , `..._MAX_BLAST_RADIUS`). Human decisions: `POST /api/v1/network/response/{id}/apply|approve|reject` .
- **Workload Containment** — Six containment profiles (FullIsolation, InternetLock, ExfiltrationLock, InvestigationMode, LateralMovementLock, timed Maintenance) enforced through Cilium, nftables, or libvirt backends. — *Quarantine a compromised pod, VM, or host while keeping DNS and essentials alive.*
  - **How:** API: `/api/v1/network/containments/*` (simulate / apply / list / release / approve / break-glass / export). Web UI: ServiceMap isolation overlay + `/embed/forensics` .
- **GuardPolicy Namespace & Cluster-wide Scope** — Guard presets (lock / lateral / exfil / quarantine) apply to a whole namespace or the entire cluster, not just one Pod or VM, reconciled by the in-cluster operator into a real `CiliumNetworkPolicy` / `CiliumClusterwideNetworkPolicy` . — *Contain lateral movement or lock down a whole namespace in one action, not one workload at a time.*
  - **How:** API: `POST /api/v1/network/guard-policies/apply` with `scope: {level: "namespace"|"clusterWide", ...}` . Operator reconciles `GuardPolicy.spec.target.kind: Namespace|ClusterWide` into the effective policy and a `WorkloadContainment` twin.

- **Forensic Pack & Incident Story** — Bundles IR evidence with a CEF sample and generates an attack narrative from correlated threat threads. — *Hand responders a ready-made evidence package, not raw logs.*
  - **How:** API: `GET /api/v1/network/forensic-pack`, `GET /api/v1/network/incident-story`. Web UI: Forensics (`/embed/forensics`), Incident Coach (`/embed/incident-coach`).
- **Root Cause Analysis** — Correlates dropped packets with active policies and offers one-click corrective CiliumNetworkPolicy fixes. — *Go from 'connection refused' to the exact offending rule instantly.*
  - **How:** Web UI: Root Cause (`/rootcause`) — apply a fix with `a`. TUI: RootCause tab. Underpinned by drop/policy correlation from `GET /api/v1/kernel/events`.

> The WorkloadContainment CRD is the durable source of truth, so containment survives restarts and control-plane loss.

7. Ask Zyra Copilot

A natural-language network copilot that investigates with real read-only tools.

- **Natural-Language Networking** — Ask questions about cluster networking in plain English and get grounded answers with an Agent, Hybrid, or Rule badge. — *Query your network the way you'd ask a teammate.*
  - **How:** API: `POST /api/v1/network/copilot` (mode `auto` / `agent` / `rule`). Web UI: Copilot (`/embed/copilot`). Engine status: `GET /api/v1/network/zeus/status`.
- **Agentic Tool Calling** — In agent mode Zeus selects from 58 read-only tools in a bounded observe-then-replan loop over the cluster's real incidents and detections. — *Answers are built from live cluster state, not model memory.*
  - **How:** Enable with `PACKETWOLF_ZEUS_AGENT_ENABLED=true` (plus an LLM key), then `POST /api/v1/network/copilot` with `mode: agent`. UI toggle: Settings → Advanced → Agentic Zeus.
- **Multi-Agent Specialist Router** — Routes hard questions to threat-hunter, policy, DNS, forensics, and containment specialists with per-conversation memory. — *The right expert handles each kind of question.*
  - **How:** Automatic within agent mode of `POST /api/v1/network/copilot` (pass `history` for per-conversation memory). Web UI: Copilot (`/embed/copilot`).
- **Gated Write Actions** — Optionally proposes gated write tools (submit policy, run playbook, apply response) that require explicit confirmation before any cluster change. — *Let the copilot act, but only after you approve.*
  - **How:** Enable with `PACKETWOLF_ZEUS_WRITE_TOOLS_ENABLED=true`; proposals surface a confirmation sheet in the Copilot (`/embed/copilot`) before any change.
- **Fail-Open Reliability** — On LLM timeout or failure, Zeus falls back to a deterministic keyword rule router. — *The copilot keeps answering even when the model is unavailable.*
  - **How:** Automatic — no configuration. `GET /api/v1/network/zeus/status` reports the active engine mode (`rule` / `hybrid` / `agent`).

> Zeus is optional and off by default — LLM and agent modes require explicit env flags or a Settings toggle.

8. Reliability Engineering

Validate resilience with real chaos, canaries, and time-travel replay.

- **Chaos Engineering** — Injects real tc-netem faults (latency, loss, corruption) via kubectl exec with circuit breakers and automatic cleanup. — *Prove resilience with genuine kernel faults, safely bounded.*
  - **How:** CLI: `netpred chaos run latency-spike --namespace default --target my-pod`, `netpred chaos list`. TUI: Chaos tab (`v` to toggle view). The web **Chaos** page only **logs** an experiment record (audit outcome `recorded`); it does not inject a fault. See Known limits.
- **Canary Deployments** — Progressive traffic shifting without sidecars, auto-promoting at 99% success and auto-rolling-back below 90%. — *Ship changes gradually and let the data decide.*
  - **How:** CLI: `netpred canary start my-deploy --weight 10`, `netpred canary promote my-deploy`. TUI: Canary tab (`p` to promote).
- **Time-Travel Replay** — Records flows to disk and replays them with VCR controls — play, pause, step, and jump to events. — *Rewind an incident and watch exactly how it unfolded.*
  - **How:** TUI: Replay tab — `t` enters time-travel mode, `Space` plays/pauses. Records to disk and replays with step/jump controls. The web Replay page keeps recording entries only (they expire after 7 days).
- **Dry-Run Simulator** — Tests policy changes against built-in scenarios with flows-affected, services-impacted, and risk scoring. — *Answer 'what would this change do' before touching production.*
  - **How:** CLI: `netpred policy simulate my-policy.yaml`. TUI: Simulator tab (`s` to run). API: `POST /api/v1/network/blast-radius`.
- **Multi-Cluster Autopilot** — Registers clusters, health-checks them, visualizes topology, and synchronizes policy across contexts. — *Run network policy and visibility across clusters from one pane.*
  - **How:** TUI: MultiCluster tab (`h` to health-check all clusters). Containment and views route across `kubectl` contexts. In the web console each cluster card is sampled through that cluster's own Hubble relay; network health, live connections, blocked flows and last sync show as not reported (null) when the API has nothing to compute them from, and the sampled-flow count is a capped sample, not a connection count.
- **Connectivity Wizard** — Step-by-step pod and service reachability checks that root-cause why traffic isn't flowing. — *Debug 'why can't A reach B' with a guided checklist.*
  - **How:** API: `POST /api/v1/troubleshoot/run`. Web UI: Connectivity Wizard (`/embed/connectivity-wizard`), Diagnostics (`/diagnostics`).

9. Compliance, Cost & Prediction

Executive posture, spend awareness, and where the cluster is trending next.

- **Compliance Lens** — Maps live posture to SOC2, PCI-DSS, HIPAA, and Zero Trust controls. — *Turn network state into audit-ready control evidence.*
  - **How:** API: `GET /api/v1/network/compliance-lens`. Web UI: Compliance Lens (`/embed/compliance-lens`).
- **CVE Posture** — Correlates runtime and egress exposure for image and CVE review. — *Prioritize the CVEs that are actually reachable over the wire.*
  - **How:** API: `GET /api/v1/network/cve-posture`. Web UI: CVE Posture (`/embed/cve-posture`).
- **Cost-Aware Egress** — Surfaces egress cost hotspots and optimization opportunities. — *Cut cloud egress spend by seeing where bytes leave.*
  - **How:** API: `GET /api/v1/network/cost-aware`. Web UI: Cost-aware Egress (`/embed/cost-aware`). Dollar figures are estimates, not billing data; where the API reports none the UI shows *not reported* rather than a made-up number.
- **Risk Trend & Forecast** — A 14-day risk ring buffer projects 'critical in ~N hours' and forecasts likely next attack targets. — *Act on where risk is heading, not just where it is.*
  - **How:** API: `GET /api/v1/network/risk-trend`, `GET /api/v1/network/forecast`, `GET /api/v1/network/attack-forecast`.
- **Network SIEM Report** — Generates an executive report with anomalies and a governance bundle. — *Hand leadership a network security summary they can read.*
  - **How:** API: `GET /api/v1/network/network-siem`. Web UI: Network SIEM (`/embed/network-siem`).
- **Digital Twin & Network Score** — Snapshots the network as a digital twin and rolls posture into a composite health score. — *Track one trendable number for overall network health.*
  - **How:** API: `GET /api/v1/network/digital-twin`, `GET /api/v1/network/network-score`. Web UI: `/embed/digital-twin`, `/embed/network-score`.

10. Surfaces & Platform

Web, terminal, CLI, and a Kubernetes-native operator over one Rust core.

Surface	What it is	Best for
Web console	90 pages, 60 panels, light and dark, phone layout	Visual investigation and reporting
TUI	18 tabs in the terminal	Fast ops on a server or over SSH
netpred CLI	10 subcommands + explain	Scripting and CI automation
Operator + CRDs	6 CRDs, kube-rs controllers	GitOps-managed network intent
REST / WebSocket API	400+ documented routes + live streams	Integrations and partner tools

- **Web Console** — 90 pages and 60 embeddable panels in the Netra style: light and dark themes, a top navigation bar, a search palette, tabs and filters kept in the URL, paged lists that say *Showing N of TOTAL*, and a phone layout. Every route sets its own page title (`<Page> · PacketWolf`) and controls carry accessible names. Leaving a page with unsaved edits asks first. — *A single pane for the whole platform, on a desk or a phone.*
  - **How:** Open the console on the API's port (9191 in the image). Search with `⌘K` / `Ctrl+K` or `/` ; `?` lists the shortcuts.
- **18-Tab TUI** — A ratatui terminal UI covering flows, connections, policies, graph, healer, chaos, canary, and kernel views. — *Full-power operations without leaving the terminal.*
  - **How:** Run `netpred tui` (or `./target/release/packetwolf`). Cycle tabs with `Tab` / `Shift+Tab` ; `?` for help; `q` to quit.
- **netpred CLI** — Ten subcommands for trace, policy, graph, chaos, canary, operator, CRDs, kernel, and explain — scriptable end to end. — *Automate and pipe network intelligence into your workflows.*
  - **How:** Run any subcommand, e.g. `netpred trace` , `netpred kernel connections` , `netpred explain` ; `netpred --help` lists all ten.
- **Kubernetes Operator** — Six CRDs and controllers using kube-rs with finalizers, status conditions, and leader election. — *Manage network intent as native Kubernetes resources.*
  - **How:** CLI: `netpred crds install` , `netpred operator start` , `netpred operator status` . Verify with `kubectl get crd | grep netpredator` .
- **Helm & Remote Deploy** — Helm chart plus deploy-remote and finish-host scripts stand up API, UI, and systemd over SSH/rsync. — *Go from clone to a running dashboard on a real host fast.*
  - **How:** `helm install packetwolf ./chart --set auth.jwtSecret=...` or `./scripts/deploy-remote.sh user@host.example.com` (→ `https://:9443`).

- **SASE-Adjacent APIs** — Ingress/egress security overviews, structured egress-gateway CRUD, and Hubble-to-CNP policy drafts. — *Cover SASE workflows teams already need without the full stack.*
  - **How:** API: `GET /api/v1/security/egress/overview`, `GET /api/v1/security/ingress/overview`, `GET /api/v1/policies/from-flows`, `POST /api/v1/egress/gateway-policies/structured`. Web UI: `/egress-security`, `/ingress-security`.

11. Container & Workload Consoles

Reach inside any workload — a Kubernetes pod, a KubeVirt VM, or a plain podman/docker container — to inspect, expose, and contain it from the browser.

- **Non-Kubernetes Container Visibility** — A lightweight node agent discovers podman, docker, and standalone containerd containers on each host and reports them to the API, so they appear as first-class workloads next to k8s pods and VMs (k8s-managed CRI containers are de-duplicated). — *See the podman/docker apps your cluster-only tools can't.*
  - **How:** Install the agent per host: `sudo ./scripts/install-node-agent.sh https://<host>:9443 <admin-api-key>` (systemd `pw-node-agent` — works on cluster nodes and standalone hosts alike). API: `GET /api/v1/containers`. Web UI: Containers (`/containers`).
- **In-Browser Container & Pod Shell** — Open an interactive shell into a running container or pod straight from the UI, PTY-bridged over a websocket (`kubectl exec` for pods; `podman` / `docker` / `crictl exec` for non-k8s containers). — *Debug a workload without hunting for the right host and exec incantation.*
  - **How:** Web UI: Containers (`/containers`) → **Shell**; Pod Guard (`/pod-guard`) top toolbar → **Shell**. API: `ws /api/v1/ws/container-exec`, `ws /api/v1/ws/exec`. Shell sessions (including VM console and VNC) are audited on open and close, capped at 3 per user and 20 in total by default, and closed after 15 idle minutes of user input and after 8 hours at most (`PW_SHELL_MAX_PER_USER`, `PW_SHELL_MAX_TOTAL`, `PW_SHELL_IDLE_SECS`, `PW_SHELL_MAX_SECS`), and closed when the user's token expires or the account is disabled or deleted.
- **VM Serial Console & VNC** — Reach a KubeVirt VM's serial console or graphical VNC framebuffer in the browser, proxied through the VMI `console` / `vnc` subresources. — *Recover or inspect a VM (login prompt, boot, GUI) with no external console tooling.*
  - **How:** Web UI: VM Guard (`/vm-guard`) top toolbar → **Console** / **VNC**. API: `ws /api/v1/ws/vm-console` (`kind=console|vnc`).
- **VM NIC binding control** — Surface each VM's NIC binding on VM Guard; switch the default NIC to **masquerade** (with VMI restart) when bridge/macvtap would leave Guard only partially enforceable. — *Make Cilium/PacketWolf control match what the guest actually uses.*
  - **How:** Web UI: VM Guard top toolbar → **Switch to masquerade** (Details shows a limited-control warn). API: `GET /api/v1/network/vms` (`interfaces`, `enforcement_coverage`, `controllable`); `POST /api/v1/vms/{ns}/{name}/network/default-nic` (`binding=masquerade`, `restart=true`).

- **Expose & De-Expose Ports** — Publish a workload port on a host port on demand and revoke it. Pods/VMs get a NodePort Service (SSH/RDP or generic TCP/UDP); non-k8s containers get an agent-run TCP/UDP proxy. — *One-click reachability for access or debugging, cleanly torn down after.*
  - **How:** Web UI: Containers / Guard top toolbar → **Ports** drawer. API: `POST` / `DELETE` `/api/v1/containers/{node}/{container}/expose`; `POST` / `DELETE` `/api/v1/pods|vms/{ns}/{name}/remote-access/expose`. Pod and VM exposure is closed by default: pass `source_cidrs` (required, non-empty) or `{"any": true}` to expose to everyone, and `ttl_minutes` between 1 and 1440 if you want an expiry.
- **Block / Allow Enforcement** — Graduated network lockdown per workload: block internet, DNS, or web egress; full isolation (quarantine); and release. Pods/VMs use CiliumNetworkPolicy; non-k8s containers use agent-applied nftables scoped to the container IP. — *Contain a suspicious workload in one click, revert just as fast.*
  - **How:** Web UI: Pod Guard / VM Guard **sticky navbar** (Lock internet / Unlock / Quarantine) and **Details** drawer for mode/TTL / Applied rules. Enforce creates a per-target `GuardPolicy` + CNP (`packetwolf-guard-<name>`) with **exclusive** ownership (Guard `egressDeny` dominates other CNP allows for that endpoint). Unlock releases GuardPolicy and the `packetwolf-contain-<name>` twin (the release asks for an optional reason). API: `POST /api/v1/network/guard-policies/apply` / `…/release` (optional body `{"reason"}`: trimmed, max 500 chars, no control characters; audited as `guard.release` and echoed in the response); also `POST /api/v1/containers/{node}/{container}/enforce` (`internet|dns|web|isolate|allow`).
- **Host-Network Tracking** — Host-networked containers (which have no container IP) resolve to the host's real NIC IPs and the ports their process actually serves, flagged with a `HOST-NET` badge; per-container enforcement is disabled for them since they share the host stack. — *A host-networked website still shows exactly where it lives and listens.*
  - **How:** Web UI: Containers (`/containers`) — a host-net row shows the host IP + serving ports and the `HOST-NET` badge.
- **Multi-Replica Ready** — Container shell/expose/enforce keep working when the API runs multiple replicas: a Redis pub/sub bus routes each operation to whichever pod holds the agent's connection (local-only when Redis is absent). — *Interactive workload access survives an HA, load-balanced API.*
  - **How:** Helm values for HA: `redis.enabled=true`, `api.service.hostPort=0`, `api.replicas>=2`.

> Golden feature: from one Containers page, shell into a podman container, expose its port, or quarantine it with nftables — the same console that manages Kubernetes pods and KubeVirt VMs.

Getting started

1. **Get the product** — Run the container image (`ghcr.io/zyvorai/zyvor-packetwolf`), or take the deployment package that comes with a trial or subscription (Helm chart, `netpred` CLI, TUI).
2. **Explore from the CLI** — Run netpred kernel connections and netpred explain to see kernel attribution and cross-layer root cause.
3. **Deploy to a Cilium cluster** — helm upgrade --install packetwolf ./chart -n packetwolf-system --create-namespace against a Cilium-powered cluster (credentials are generated on first install).
4. **Open the dashboard** — Open the web console on the API's port (9191 in the container image, 9443 for a remote host deploy) to reach Flows, Service Map, and Ask Zyra.
5. **Turn on the intelligence you want** — Enable custom eBPF probes (NETPRED_EBPF_AUTO=1), deep L7 inspection per workload, and optionally the Zyra copilot via env flags.

> **Good to know:** PacketWolf targets Cilium-powered Kubernetes clusters and reads Cilium's pinned kernel maps via bpftool or kubectl exec into cilium-agent. Custom eBPF probes are opt-in and require CAP_BPF; probe phases 6-8 (flow-metering enrichment, Cilium drop enrichment, security baseline) run in userspace today, and the data-plane view reports each phase's attach status honestly. L7 detectors such as NXDOMAIN/DGA and HTTP anomaly detection stay inert until deep-inspection L7 visibility is enabled for a workload, so they never false-alarm on L3/L4-only clusters. The Ask Zyra copilot's LLM, agent, and write-tool modes are off by default and require explicit configuration; write actions and high-impact containments are approval-gated. GeoIP and threat-intel enrichment are optional and depend on supplied MaxMind and feed data. Some Cilium CLI subcommands vary by release, and cluster-wide endpoint views may require aggregating across nodes.

Known limits

What this build does not do, so you can plan around it. Items marked *record-only* store an entry and write an audit record but do not change the cluster.

Record-only or flag-only actions

- **Healer fixes** are *recorded*, not applied (see Self-Healing Engine).
- **Packet capture, traffic mirror, chaos and replay** web entries are *record-only*: the API stores them, they expire after 7 days, and each kind holds at most 200 live entries (new ones are refused with 429 until you delete some). Capture does not start a capture, a mirror rule does not start mirroring (apply it with your CNI or mesh), and a chaos entry injects no fault.
- **Drift guard** only flags: it never reverts a resource (`remediated` is always 0). Resources marked for revert are for your ArgoCD or Flux sync to restore.
- **Tracing-policy simulate** is a server-side dry run.

Not supported from PacketWolf

- **ClusterMesh connect** returns 501; connect peers with `cilium clustermesh connect`.
- **Change rollback** returns 501: the change log records observed events, not snapshots; revert through GitOps or kubectl.

Estimates and gaps in reporting

- Cost figures and mesh status are marked estimated or not reported; insight scores appear only when the API reports a posture score, otherwise the page says *No data*.
- Multi-cluster health, connection and sync figures are null when they cannot be computed. This endpoint tracks no per-cluster sync.
- `GET /api/v1/flows` reads only the newest flows Hubble serves, so `total` can be a lower bound (`total_is_lower_bound`). Flow ids are positional in that window; the ids a list returns stay resolvable for about five minutes.
- The audit log read is capped (`truncated`, `scan_limit`) and entries are kept for 30 days.

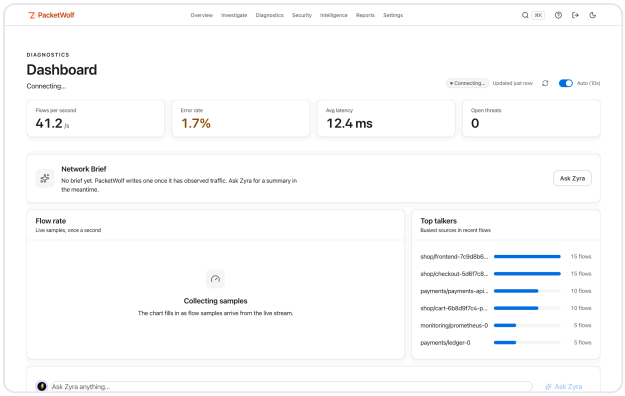
Verification gaps

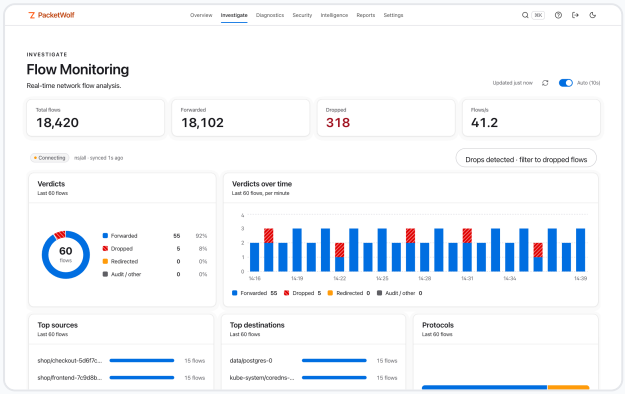
- The hardening and honesty changes in this edition (namespace scoping, approvals, logout revocation, the `kubectl` guard, forwarded-address handling) are covered by automated unit, route-behaviour and UI tests, including a test that calls every route as a scoped account. Live-cluster end-to-end verification is still outstanding. In particular the admission webhook with a real API server, WebSocket scoping, shell caps and idle timeouts under load, and the Templates apply path against a live Cilium cluster still need a live-cluster pass before you rely on them.
- **Node cordon, drain and uncordon** (opt-in, `rbac.nodeMaintenance.enabled`) and the **Tetragon** paths (events, tracing policies, runtime views) have been exercised against mocks and fake command-line tools, not against a live cluster. Trial them on a non-production cluster first.
- Cross-replica effects (a logout or role change on one replica reaching another) rely on Redis; without Redis they are per replica.
- Record-only features have no backing implementation yet (real capture, real mirroring, real fault injection, healer execution, auto-remediating drift guard, ClusterMesh peer management).

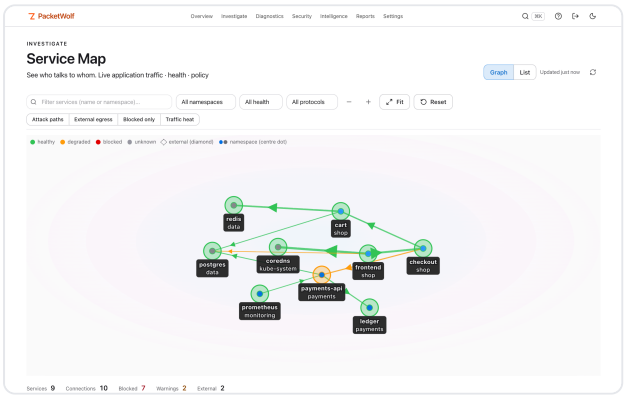
Screenshots

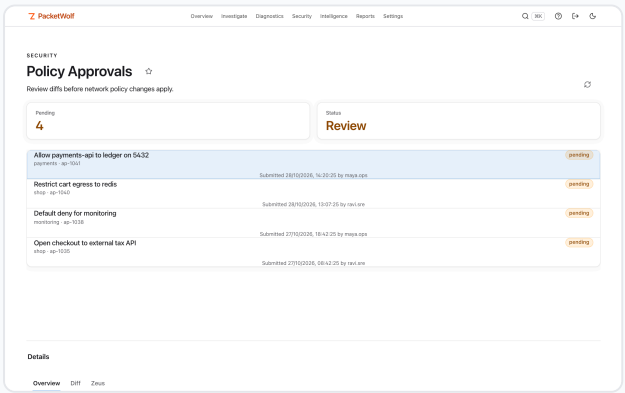
Taken from the current console against synthetic sample data. Files are in `docs/screenshots/`.

Light

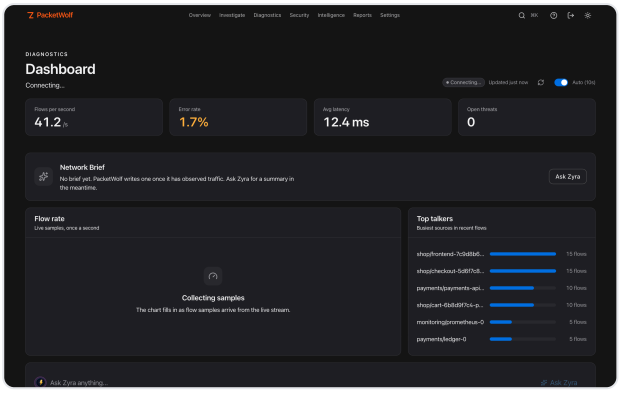

PacketWolf
DIAGNOSTICS
Dashboard
Connecting...
Flows per second
41.2
Error rate
1.7%
Avg latency
12.4 ms
Open threats
0
Network Brief
No brief yet. PacketWolf writes one once it has observed traffic. Ask Zyra for a summary in the meantime.
Ask Zyra
Flow rate
Live samples, once a second
Collecting samples
The chart fills in as flow samples arrive from the live stream.
Top talkers
Busiest sources in recent flows
Ask Zyra anything...

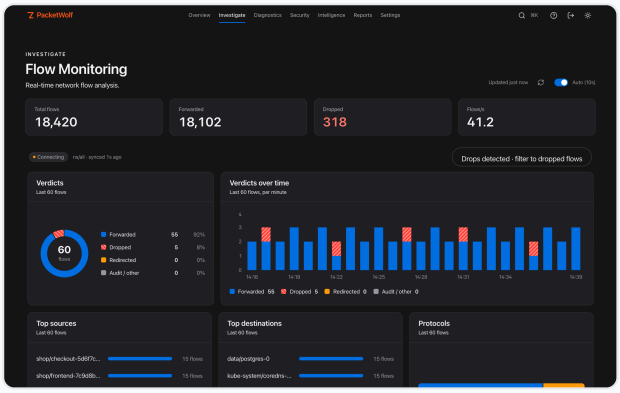

PacketWolf
INVESTIGATE
Flow Monitoring
Real-time network flow analysis.
Total flows
18,420
Forwarded
18,102
Dropped
318
Flows/s
41.2
Drops detected · filter to dropped flows
Verdicts
Last 60 flows
60
flows
Forwarded 55 92%
Dropped 5 8%
Redirected 0 0%
Audit / other 0 0%
Verdicts over time
Last 60 flows, per minute
Forwarded 55 Dropped 5 Redirected 0 Audit / other 0
Top sources
Last 60 flows
shop/checkout-5d6f7c... 15 flows
shop/frontend-7c9d8b... 15 flows
Top destinations
Last 60 flows
data/postgres-0 15 flows
kube-system/coredns-... 15 flows
Protocols
Last 60 flows

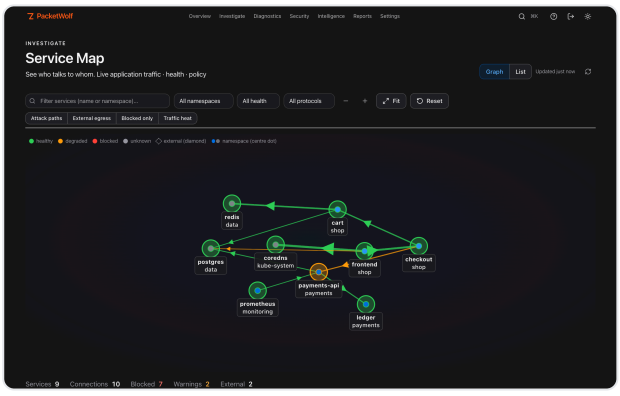

PacketWolf
INVESTIGATE
Service Map
See who talks to whom. Live application traffic · health · policy
Graph
List
Updated just now
All namespaces
All health
All protocols
Fit
Reset
Attack paths
External egress
Blocked only
Traffic heat
healthy
degraded
blocked
unknown
external (diamond)
namespace (centre dot)
redis
data
cart
shop
postgres
data
coredns
kube-system
frontend
shop
checkout
shop
payments-api
payments
prometheus
monitoring
ledger
payments
Services 9
Connections 10
Blocked 7
Warnings 2
External 2

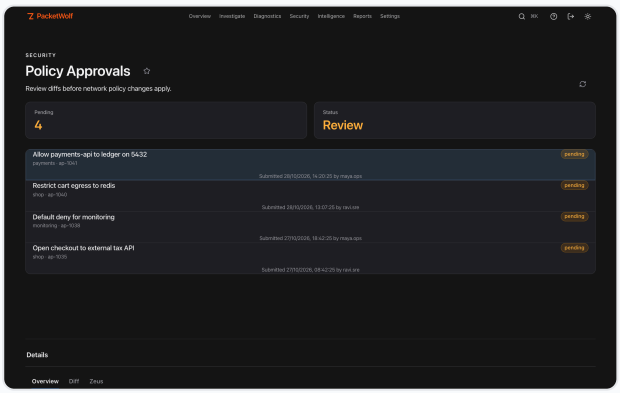

PacketWolf
SECURITY
Policy Approvals
Review diffs before network policy changes apply.
Pending
4
Status
Review
Allow payments-api to ledger on 5432
payments · ap-1041
pending
Restrict cart egress to redis
shop · ap-1042
pending
Default deny for monitoring
monitoring · ap-1038
pending
Open checkout to external tax API
shop · ap-1035
pending
Details
Overview
Diff
Zeus

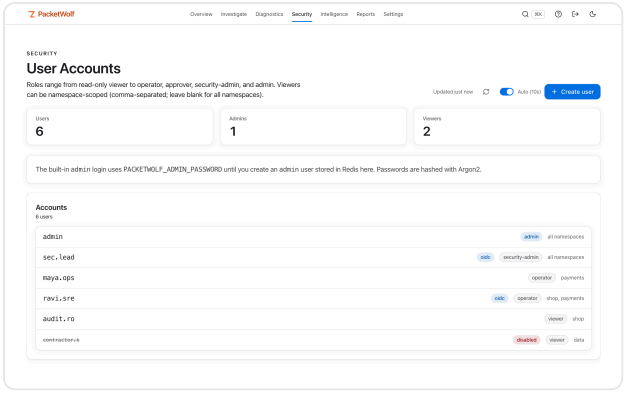
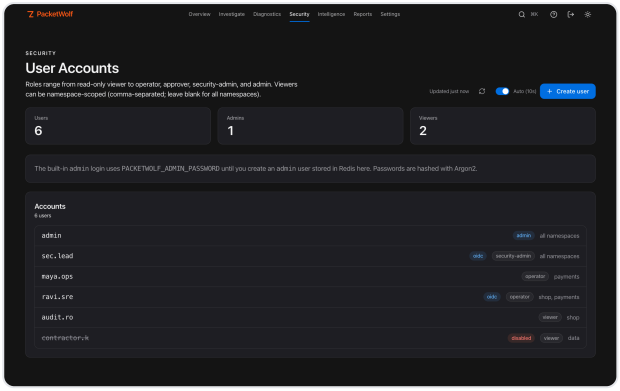
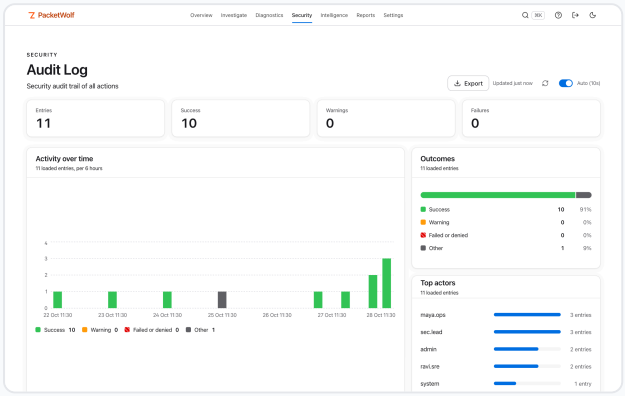
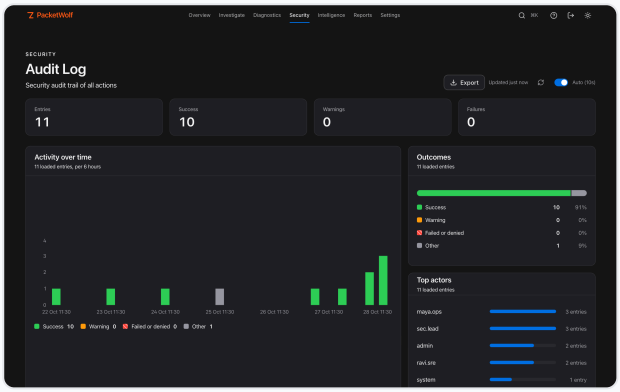
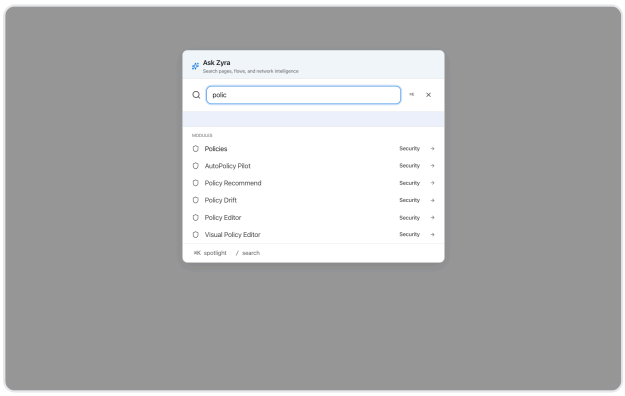
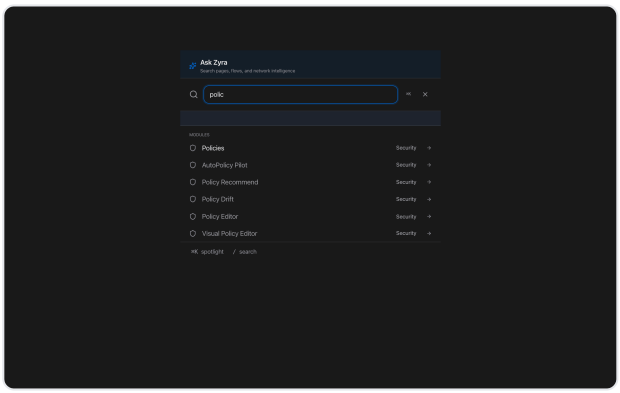
Dark


PacketWolf
DIAGNOSTICS
Dashboard
Connecting...
Flows per second
41.2
Error rate
1.7%
Avg latency
12.4 ms
Open threats
0
Network Brief
No brief yet. PacketWolf writes one once it has observed traffic. Ask Zyra for a summary in the meantime.
Ask Zyra
Flow rate
Live samples, once a second
Collecting samples
The chart fills in as flow samples arrive from the live stream.
Top talkers
Busiest sources in recent flows
Ask Zyra anything...


PacketWolf
INVESTIGATE
Flow Monitoring
Real-time network flow analysis.
Total flows
18,420
Forwarded
18,102
Dropped
318
Flows/s
41.2
Drops detected · filter to dropped flows
Verdicts
Last 60 flows
60
flows
Forwarded 55 92%
Dropped 5 8%
Redirected 0 0%
Audit / other 0 0%
Verdicts over time
Last 60 flows, per minute
Forwarded 55 Dropped 5 Redirected 0 Audit / other 0
Top sources
Last 60 flows
shop/checkout-5d6f7c... 15 flows
shop/frontend-7c9d8b... 15 flows
Top destinations
Last 60 flows
data/postgres-0 15 flows
kube-system/coredns-... 15 flows
Protocols
Last 60 flows

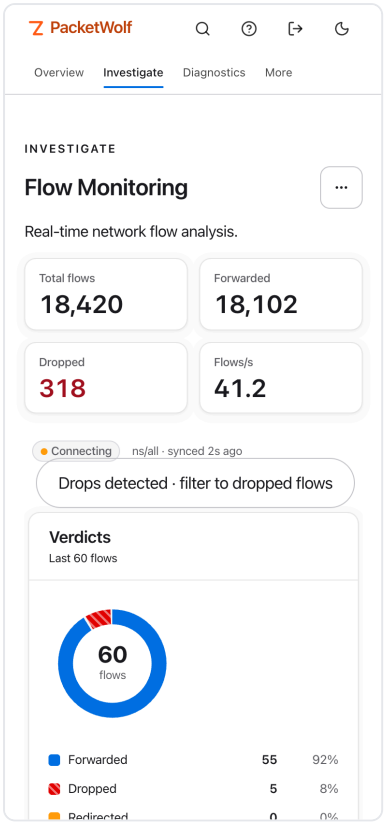

PacketWolf
INVESTIGATE
Service Map
See who talks to whom. Live application traffic · health · policy
Graph
List
Updated just now
All namespaces
All health
All protocols
Fit
Reset
Attack paths
External egress
Blocked only
Traffic heat
healthy
degraded
blocked
unknown
external (diamond)
namespace (centre dot)
redis
data
cart
shop
postgres
data
coredns
kube-system
frontend
shop
checkout
shop
payments-api
payments
prometheus
monitoring
ledger
payments
Services 9
Connections 10
Blocked 7
Warnings 2
External 2


PacketWolf
SECURITY
Policy Approvals
Review diffs before network policy changes apply.
Pending
4
Status
Review
Allow payments-api to ledger on 5432
payments · ap-1041
pending
Restrict cart egress to redis
shop · ap-1042
pending
Default deny for monitoring
monitoring · ap-1038
pending
Open checkout to external tax API
shop · ap-1035
pending
Details
Overview
Diff
Zeus

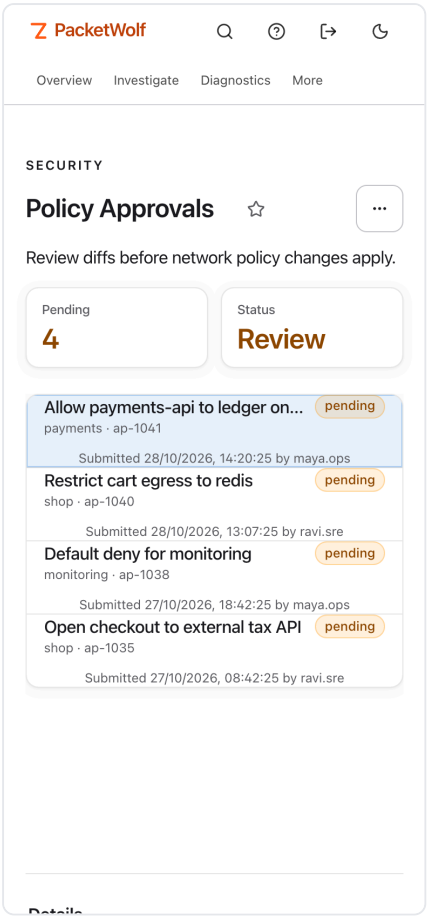
Light	Dark
	
	
	

On a phone (390 px wide):

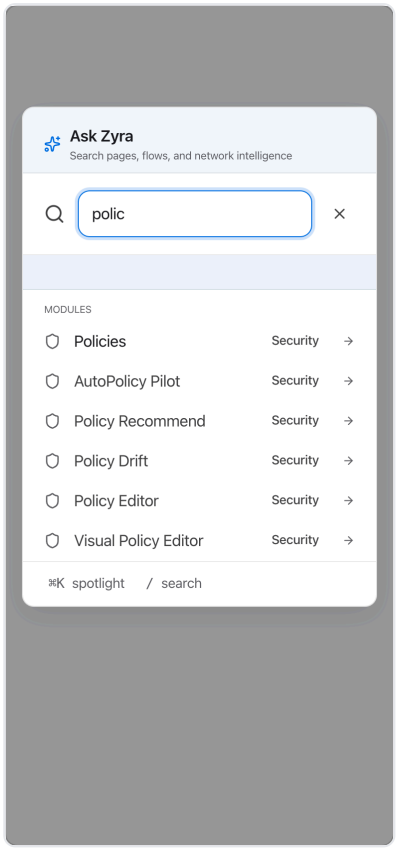
Flows

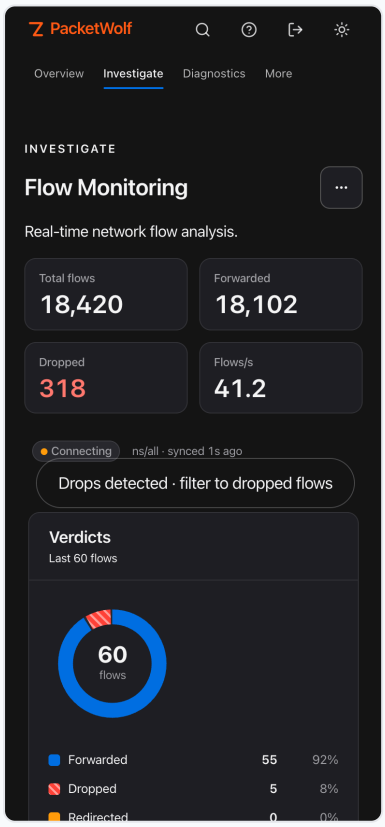


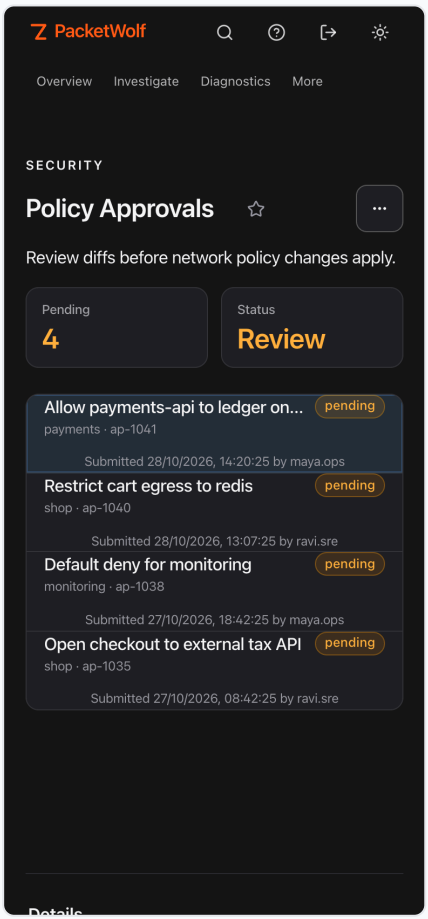
Policy Approvals

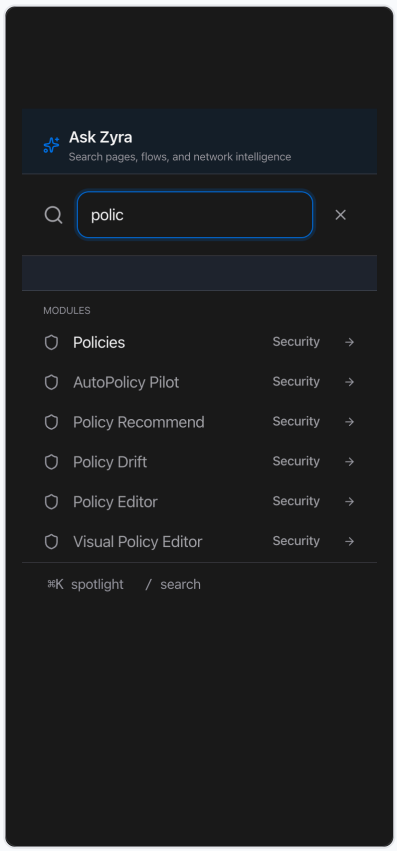


Search









*PacketWolf is developed by ZyvorAI Labs. Contact **info@zyvor.dev** · Proprietary & Confidential.*